

Лабораторная работа №8: Перечисления (Enum), сохранение и загрузка данных, взаимодействие между объектами

Цель. Научиться:

- применять перечисления (enum class) для упрощения логики выбора;
- реализовывать взаимодействие между классами проекта;
- сохранять и загружать данные;
- развивать и расширять игровой проект на Kotlin с учётом ранее созданных классов.

В данной лабораторной работе мы реализуем **расширенный функционал NPC**, добавив нового персонажа — **Торговца (Trader)**, который сможет:

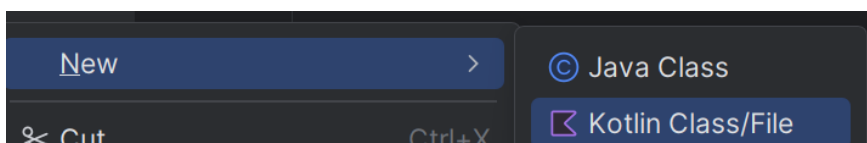
- выдавать **различные типы квестов** в зависимости от выбора игрока;
- продавать **питомцев** из модуля **pets**;
- взаимодействовать с героем (**Hero**) и сохранять полученные квесты в файл.

Также в проект будет добавлен новый класс **QuestType**, реализованный с помощью **enum class**.

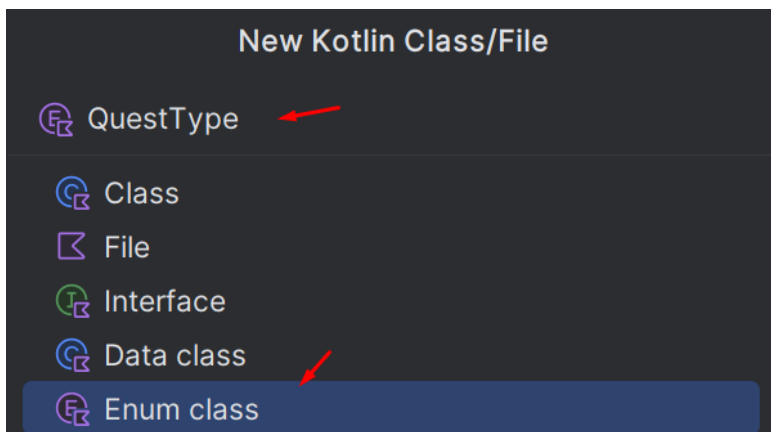
Шаг 1: Создание перечисления QuestType

В программировании часто возникает необходимость ограничить значения некоторого поля заранее определённым набором. Например, типы квестов: сопровождение, доставка, битва и т.д. Если использовать обычные строки, то легко допустить ошибку: опечататься или указать несуществующее значение. Чтобы избежать подобных ошибок и сделать код более читаемым и безопасным, в языке Kotlin используется специальный тип — **enum class**.

В пакете **world** создайте новый класс (New – Kotlin Class/File):



Введите имя **QuestType**, но в качестве типа выберите **Enum class**:



У нас с вами созданся пустой **enum class**:

```
package world

enum class QuestType { new *
}

```

Правила написания enum:

- Названия констант в enum class по соглашению пишутся **ЗАГЛАВНЫМИ буквами**, через подчеркивание (`_`) при необходимости.
- Перечисления по умолчанию являются **синглтонами** — у каждой константы один экземпляр.
- enum может содержать **собственные параметры, свойства и методы**.
- Каждый элемент может содержать своё **значение** (например, описание) и даже переопределять методы.

В нашем проекте мы хотим классифицировать квесты. Мы не хотим, чтобы программисты случайно ввели строку "elimination" (с ошибкой) или "битва" вместо "BOSSFIGHT". Создаём перечисление:

```
enum class QuestType(val description: String) { 5 Usages
    DELIVERY(description = "Доставка предмета"),
    ELIMINATION(description = "Устранение противника"),
    ESCORT(description = "Сопровождение персонажа"),
    EXPLORE(description = "Исследование новой территории"),
    BOSSFIGHT(description = "Битва с боссом")
}

```

- **enum class QuestType(...)** - Объявление перечисления QuestType;
- **DELIVERY("Доставка предмета")** - Один из возможных вариантов, константа с параметром — описанием типа квеста;
- **val description: String** - Свойство, которое будет содержать строковое описание, переданное в скобках.

Шаг 2: Обновление класса Quest

Наша цель - добавить новое свойство **questType** (типа **QuestType**) в класс **Quest**, чтобы у каждого квеста был чётко заданный тип: доставка, бой с боссом, сопровождение и т.д.

Это позволит:

- ✓ Систематизировать квесты;
- ✓ Упростить фильтрацию и генерацию заданий по категориям;
- ✓ Повысить читаемость и масштабируемость проекта.

Добавим новое свойство в класс **Quest**:

```
class Quest( 3 Usages  ⤵ Denis *  
    title: String,  
    val duration: Int,  
    reward: Int,  
    val difficulty: String,  
    val questType: QuestType // <-- новое поле  
) : Mission(title, reward) {
```

Теперь при создании квеста мы будем обязательно указывать его тип — это избавит от магических строк.

Обновим метод **describe()**, чтобы отобразить новый параметр:

```
override fun describe() { ⤵ Denis *  
    println("Квест '$title' на $duration часов, сложность:  
        $difficulty, награда: $reward золотых")  
    println("Тип квеста: ${questType.description}") // добавляем вывод  
}
```

И метод **printInfo()**:

```
fun printInfo() { ⤵ Denis *  
    println("Название квеста: $title")  
    println("Тип квеста: ${questType.description}")  
    println("Время выполнения: $duration ч.")  
    println("Награда: $reward золотых")  
    println("Уровень сложности: $difficulty")  
}
```

Далее в файле **Guild.kt** создадим квест:

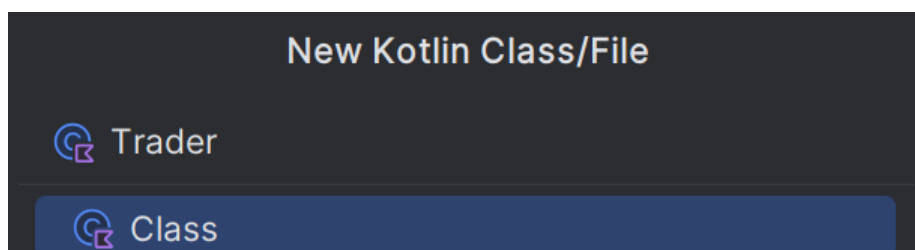
```
fun main() {  ⤴ Denis *
    val escortQuest = Quest(
        title = "Сопроводи торговца до деревни",
        duration = 4,
        reward = 120,
        difficulty = "Средний",
        questType = QuestType.ESCORT
    )
    escortQuest.printInfo()
}
```

Шаг 3: Создание класса Торговца (Trader) с выдачей квестов

Наша цель создать класс торговца, который будет:

- Владеть списком квестов (объекты класса Quest из пакета world)
- Предоставлять эти квесты героям по запросу
- Позволять показывать доступные квесты в консоли
- В дальнейшем — сохранять и загружать квесты из файла (будет реализовано в следующих шагах)

В пакете **characters** создайте новый класс **Trader**:



Для работы с квестами мы должны в начале импортировать пакет:

```
package characters
import world.Quest
```

В качестве полей определим имя торговца **name** и список квестов, которые может выдавать торговец **quests**. Они будут храниться как **MutableList**, чтобы можно было добавлять новые квесты:

```
class Trader( new *
    val name: String,
    private val quests: MutableList<Quest> = mutableListOf()
) {

}
```

Создадим следующие методы:

Метод **showAvailableQuests()** — выводит в консоль список доступных квестов с краткой информацией.

```
fun showAvailableQuests() { new *
    println("Доступные квесты от $name:")
    quests.forEachIndexed { index, quest ->
        println("${index + 1}. ${quest.title} (${quest.questType}
            .description}) - Награда: ${quest.reward} золота")
    }
}
```

Метод **giveQuest()** — по индексу выдаёт конкретный квест (или null, если индекс неверный).

```
fun giveQuest(index: Int): Quest {
    return quests[index - 1]
}
```

Метод **addQuest()** — позволяет добавить новый квест в список торговца.

```
fun addQuest(quest: Quest) { new *
    quests.add(quest)
    println("Квест '${quest.title}' добавлен в список $name.")
}
```

В файле **Person.kt** импортируем нужные пакеты:

```
import world.Quest
import world.QuestType
```

Затем посмотрим пример работы класса:

```
fun main() {  👤 Denis *
    val trader = Trader(name = "Ральф")

    trader.addQuest(Quest(title = "Собрать травы", duration = 2,
        reward = 50, difficulty = "Лёгкий", QuestType.DELIVERY))
    trader.addQuest(Quest(title = "Убить волков", duration = 3,
        reward = 100, difficulty = "Средний", QuestType.ELIMINATION))

    trader.showAvailableQuests()

    val selectedQuest = trader.giveQuest(index = 1)
    selectedQuest.describe()
}
```

Шаг 4: Работа с файлами в Kotlin (через пакет files)

Во время работы RPG-игры может понадобиться сохранять:

- Список доступных квестов;
- Состояние героя;
- Прогресс игрока;
- Журнал событий и лог боя и т.д.

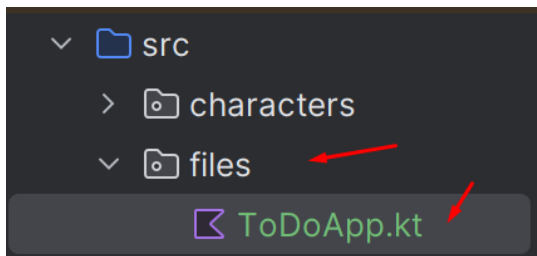
Чтобы при следующем запуске программы данные не терялись, их нужно **сохранять во внешний файл**, а затем при необходимости **считывать обратно**.

Kotlin предлагает простой и лаконичный способ работы с файлами через класс `java.io.File`.

Основные методы работы с файлами:

Метод	Назначение
writeText(text)	Полностью перезаписывает файл новым текстом
appendText(text)	Добавляет текст в конец файла, не удаляя существующее
readText()	Считывает всё содержимое файла в виде одной строки
readLines()	Считывает файл построчно и возвращает список строк

В папке нашего проекта давайте создадим простой Kotlin-файл **ToDoApp.kt** в пакете **files**:



Создаём точку входа в программу, метод **main**:

```
1 package files
2
3 fun main() { new *
4
5 }
```

Создадим простой файл:

```
fun main() { new *
    val file = File("demo.txt")
}
```

У нас отсутствует необходимый импорт, наведите на **File**, и нажмите на импорт:

```
fun main() { new *
    val file = File("demo.txt")
}
```

Unresolved reference 'File'.

Import class 'File' Alt+Shift+Enter

В начале добавится нужный пакет:

```
import java.io.File
```

В начале запишем строку «Привет!» методом `writeText()` и выведем текущее содержание файла через `readText()`:

```
val file = File(pathname = "demo.txt")
println("Создаём или перезаписываем файл...")
file.writeText(text = "Привет!\n")
println("Текущее содержимое файла:")
println(file.readText())
```

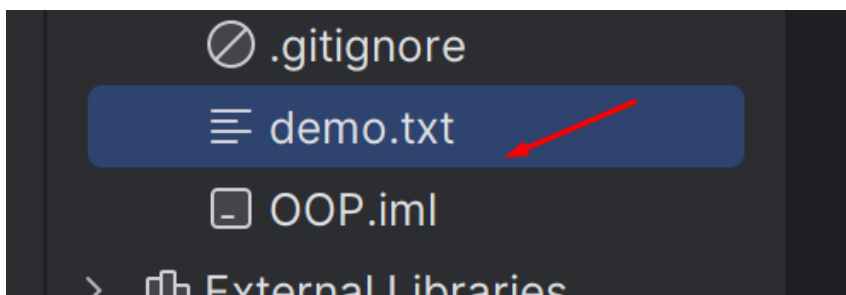
Затем добавим новую строку "Это новая строка!" методом `appendText()`. И выведем содержимое файла с двумя строками:

```
println("Добавим новую строку с помощью appendText...")
file.appendText(text = "Это новая строка!\n")
println("Строка добавлена.")
println("Текущее содержимое файла:")
println(file.readText())
```

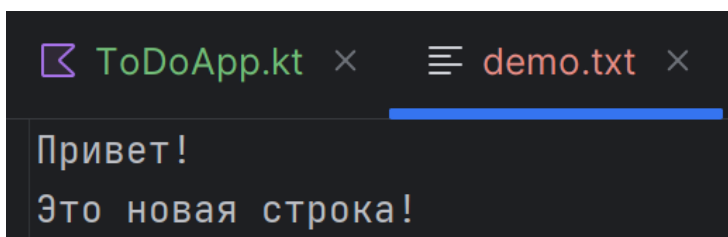
Затем прочитаем файл в список строк `readLines()` и через цикл `for` выведем каждую строку с её номером (индекс + 1):

```
println("Прочитаем файл построчно (readLines):")
val lines = file.readLines()
for ((index, line) in lines.withIndex()) {
    println("${index + 1}: $line")
}
```

Обратите внимание, у нас в папке с проектом создался файл `demo.txt`:



Откройте его, и посмотрите содержимое:



Очистите код внутри метода `main`, оставив только создание файла:

```
fun main() { new *
    val file = File(pathname = "todo.txt")
}
```

Теперь давайте реализуем простое приложение, которое:

1. Позволяет пользователю вводить задачи.
2. Сохраняет их в файл **toDo.txt**.
3. Выводит список всех задач при завершении.

У нас с вами уже есть объект файла, в который мы будем сохранять

```
val file = File(pathname = "toDo.txt")
```

Давайте добавим приветственное сообщение:

```
val file = File(pathname = "toDo.txt")  
println("Добро пожаловать в To-Do приложение!")
```

Для того, чтобы пользователь мог вводить много новых задач, нам нужно создать бесконечный цикл:

```
while (true) {  
  
}
```

Добавим запрос для добавления новой задачи и сохранение его в переменную **userInput()**:

```
while (true) {  
    print("Введите задачу (или 0 для выхода): ")  
    val userInput = readln()
```

Теперь нам нужно сделать проверку на выход, если пользователь введёт 0, то нужно прервать цикл, в ином случае нужно добавить задачу и сделать переход на новую строку. Также добавим вывод подтверждения сохранения:

```
if (userInput == "0") {  
    break  
} else {  
    file.appendText(text = "$userInput\n")  
    println("Задача сохранена!")  
}
```

И после цикла выведем список наших задач:

```

fun main() { new *
    val file = File(pathname = "ToDo.txt")
    println("Добро пожаловать в To-Do приложение!")

    while (true) {...}

    println("\nВаш список задач:")
    println(file.readText())
}

```

Шаг 5: Создание простого текстового менеджера задач (To-Do) с сохранением в файл

Сейчас мы решим с вами сложную задачу, используя все ранее накопленные знания. Мы создадим следующее приложение:

```

=== МЕНЮ ЗАДАЧ ===
1 - Добавить задачу
2 - Показать все задачи
3 - Удалить задачу
0 - Выход
Выберите действие:

```

Его возможности:

✓ Добавления новых задач

```

Выберите действие: 1
Введите новую задачу: Поучить Kotlin
Задача добавлена.

```

✓ Просмотра всех задач

```

Выберите действие: 2
Все задачи:
1. Поучить Kotlin

```

✓ Удаления задач по номеру

```
Выберите действие: 3
Задачи для удаления:
1. Поучить Kotlin
Введите номер задачи для удаления: 1
Задача "Получить Kotlin" удалена.
```

✓ **Выхода** из программы

```
Выберите действие: 0
Выход из программы.
```

Наша цель:

Научиться:

- создавать простое меню взаимодействия с пользователем;
- использовать `when` для обработки ввода;
- записывать и считывать данные из файла (`File`);
- реализовывать добавление, просмотр и удаление задач;
- использовать циклы и коллекции (`MutableList`).

1. У нас имеется следующий код:

```
fun main() { new *
    val file = File(pathname = "todo.txt")
}
```

2. Создадим бесконечный цикл меню.

```
while (true) {

}
```

- Мы используем `while (true)` для создания бесконечного цикла.
- Это позволяет пользователю выполнять действия в программе, пока он сам не выберет выход.

3. Добавим вывод стандартных задач через println():

```
while (true) {  
    println("\n=== МЕНЮ ЗАДАЧ ===")  
    println("1 - Добавить задачу")  
    println("2 - Показать все задачи")  
    println("3 - Удалить задачу")  
    println("0 - Выход")  
    print("Выберите действие: ")  
}
```

4. Добавим обработку пользователя через when:

```
when (readln()) {  
    "1" -> { /* Добавление задачи */ }  
    "2" -> { /* Просмотр задач */ }  
    "3" -> { /* Удаление задачи */ }  
    "0" -> { /* Выход */ }  
    else -> println("Некорректный ввод. Попробуйте снова.")  
}
```

В блоке else мы обработали некорректный ввод. Если пользователь ввёл не 1, 2, 3 или 0 – программа сообщает об ошибке.

5. Обработка выбора (1 - Добавить задачу)

```
when (readln()) {  
    "1" -> {  
        print("Введите новую задачу: ")  
        val task = readln()  
        file.appendText(text = "$task\n")  
        println("Задача добавлена.")  
    }  
}
```

- Пользователь вводит текст задачи.
- **appendText** добавляет задачу в конец файла.
- Каждая задача записывается с новой строки.

6. Обработка выбора (2 - Показать все задачи)

```
"2" -> {
    println("Все задачи:")
    if (file.exists() && file.readLines().isNotEmpty()) {
        file.readLines().forEachIndexed { index, task ->
            println("${index + 1}. $task")
        }
    } else {
        println(" ! Список задач пуст.")
    }
}
```

- Проверяем, существует ли файл и есть ли в нём задачи.
- **readLines()** считывает все строки из файла.
- **forEachIndexed** позволяет вывести задачи с номерами.

7. Обработка выбора (3 - Удалить задачу)

Считываем все задачи в изменяемый список **tasks**:

```
"3" -> {
    val tasks = file.readLines().toMutableList()
```

Добавим блок if/else. Если список задач будет пуст, то выведем:

```
"3" -> {
    val tasks = file.readLines().toMutableList()
    if (tasks.isEmpty()) {
        println("Список задач пуст.")
    } else {

    }
}
```

Далее показываем задачи с номерами, чтобы пользователь мог выбрать нужную:

```

} else {
    println("Задачи для удаления:")
    tasks.forEachIndexed { index, task ->
        println("${index + 1}. $task")
    }
}

```

Продолжаем писать в блоке else. Считываем номер задачи, которую пользователь хочет удалить:

```

} else {
    println("Задачи для удаления:")
    tasks.forEachIndexed { index, task ->
        println("${index + 1}. $task")
    }
    print("Введите номер задачи для удаления: ")
    val num = readln().toIntOrNull()
}

```

- toIntOrNull() защищает от ошибок, если введено не число.

Далее добавим проверку на корректный ввод:

```

} else {
    println("Задачи для удаления:")
    tasks.forEachIndexed { index, task ->
        println("${index + 1}. $task")
    }
    print("Введите номер задачи для удаления: ")
    val num = readln().toIntOrNull()
    if (num == null || num !in 1..tasks.size) {
        println("Некорректный номер.")
    } else {
    }
}

```

Внутри блока else удалим выбранную задачу, очистим файл и запишем оставшиеся задачи заново:

```

if (num == null || num !in 1..tasks.size) {
    println("Некорректный номер.")
} else {
    val removed = tasks.removeAt(index = num - 1)
    file.writeText(text = "") // очищаем
    tasks.forEach { file.appendText(text = "$it\n") }
    println("Задача \"$removed\" удалена.")
}

```

Итоговый блок:

```

"3" -> {
    val tasks = file.readlines().toMutableList()
    if (tasks.isEmpty()) {
        println("Список задач пуст.")
    } else {
        println("Задачи для удаления:")
        tasks.forEachIndexed { index, task ->
            println("${index + 1}. $task")
        }
        print("Введите номер задачи для удаления: ")
        val num = readln().toIntOrNull()
        if (num == null || num !in 1..tasks.size) {
            println("Некорректный номер.")
        } else {
            val removed = tasks.removeAt(index = num - 1)
            file.writeText(text = "") // очищаем
            tasks.forEach { file.appendText(text = "$it\n") }
            println("Задача \"$removed\" удалена.")
        }
    }
}

```

8. Обработка выбора (0 - Выход)

```
"0" -> {  
    println("Выход из программы.")  
    break  
}
```

- **break** завершает бесконечный цикл, и программа заканчивается.

Самостоятельные задания

Задание 1: Работа с квестами

- Создайте несколько объектов Quest, используя разные значения QuestType.
- Используйте метод describe() и printInfo() для каждого.
- Попробуйте создать функцию, которая выводит только квесты типа EXPLORE.

Задание 2: Работа с системой ввода-вывода

Цель: Создать класс Trader, который умеет хранить список квестов, добавлять новые квесты, показывать список квестов и выдавать (удалять) квест по номеру. Реализовать в классе метод start(), который запускает консольное меню с выбором действий и работает с пользователем в цикле.

Требования:

- Хранить квесты в списке MutableList<Quest>.
- Методы:
 - addQuest(quest: Quest) — добавляет квест в список.
 - showAvailableQuests() — выводит список всех квестов с индексами.
 - giveQuest(index: Int): Quest? — возвращает квест по индексу, или null если индекс некорректный.
 - removeQuest(index: Int) — удаляет квест из списка по индексу.
 - start() — запускает цикл с меню: добавить квест, показать квесты, взять квест, выйти.

Пример реализации:

Выберите действие:

- 1 - Добавить квест
- 2 - Показать все квесты
- 3 - Взять квест (удалить из списка)
- 0 - Выйти

Ваш выбор: 1

Введите название квеста: Сбор трав

Введите длительность (часы): 3

Введите награду (золото): 450

Введите сложность: лёгкий

Введите тип квеста (DELIVERY, ELIMINATION, ESCORT, EXPLORE, BOSSFIGHT): EXPLORE

Квест 'Сбор трав' добавлен.